Q : Define Compiler. What is the important role of a compiler?

Compiler : A program that can read a program in one language (source language) and translate it into an equivalent program in another language (target language).

Important role of a compiler :

An important role of the compiler is to report any errors in the source program that it detects during the translation process.

Q : Describe the phases of compilation/compiler.

The phases of compilation :

1.Lexical analysis : Lexical analysis, also called lexeme or scanning, receives a stream of characters from the source program and groups them into tokens.

Example :

```
int main()
{
   int a = 10;
   return 0;
}
```

If we write like that

```
int main()
{
   int 1 = 10;
   return 0;
}
```

compiler will show a lexical error. Because identifiers never can be numbers.

2.Syntactic analysis : Syntax analysis, also called parsing, receives a stream of tokens from the lexeme and groups them into phrases that match specified grammatical patterns.

Example :

```
int main()
{
   string s = “Hello”;
   return 0;
}
```

If we write like that

```
int main()
{
   string s = “Hello;
   return 0;
}
```

Compiler will show a syntactic error. Because double quotation is not right.

3.Semantic analysis : Semantic analysis traverses the abstract syntax tree, checking that each node is appropriate for its context, i.e., it checks for semantic errors. It outputs a refined abstract syntax tree.

Example :

```
int main()
{
   int s = “Hello”;
   return 0;
}
```

It will show a semantic error. In this code keywords, identifiers, operator and semicolon everything is okay but assigned value “Hello” is not matching with the keywords “int”.

4.Intermediate code generation : Intermediate code generation represents the semantics of a program,

but is machine-independent.

Example :

```
int main()
{
   int a = 5, b = 10;
   int c = a + b;
   return c;
}
```

5.Optimization : Optimization reviews the code, looking for ways to reduce the number of operations and the memory requirements.

Example :

```
int main()
{

   int c = 5 + 10;
   return c;
}
```

6.Machine code generation : The machine code generation receives the (optimized) intermediate code.

It produces either machine code for a specific machine, or assembly code for a specific machine and assembler.

If it produces assembly code, then an assembler is used to produce the machine code.

Example :

```
MOV R1, #15
MOV R0, R1
RET
```

Important terms :

1.Intermediate code generation marks the boundary between the front end and the back end.

2.The front end is language-specific and machine-independent.

3.The back end is machine-specific and language-independent.

Q : Define Token and Abstract Syntax Tree.

Token : A token is a smallest meaningful group symbols.

Abstract Syntax Tree : The output of the parser is an abstract syntax tree representing the syntactical structure of the tokens.

Q : Define context-free grammar(CFG).

CFG : Context-free grammar is formally defined by 4-tuple G=(V,T,P,S) where

V describes finite set of variables.

T describes finite set of terminals.

P describes production rules.

S describes start variable.

Q : Define derivation & parse tree.

Derivation : A derivation is a sequence of production rule applications used to generate a string from a Grammar's start symbol.

Parse Tree : A parse tree is a tree representation of the syntactic structure of a string according to a given Grammar.

Q : Define leftmost derivation & rightmost derivation.

Leftmost Derivation : A leftmost derivation is one that replaces non-terminals from left to right.

Rightmost Derivation : A rightmost derivation is one that replaces non-terminals from right to left.

Q : What do you mean by ambiguity?

Ambiguity : Ambiguity is that where exists more than one derivation for any string that belongs to Grammar.

থিওরি আর চোখে পড়তেছে না। বাকি যা আসবে সব এনালাইটিকেল। ডেরিভেশন কেমনে করে ঐগুলো একটু দেখে নে। আগামীকাল লাস্ট মাইর হবে ইন শা আল্লাহ।

Solved By Mr. Pirate-3168 (5EM)